



Desenvolvimento de Aplicação com integração do orquestrador de recursos Kubernetes

Licenciatura em Engenharia Informática

João Bartolomeu Duarte

Rodrigo Caldeira Carriça

Leiria, Julho de 2025

Índice

1. Lista de Siglas e Acrónimos	4
2. Resumo	5
3. Abstract	6
4. Introdução	7
4.1. Containers vs Virtual Machines	7
4.1.1. Vantagens da Utilização de Containers	9
4.1.2. Desvantagens da Utilização de Containers	9
4.2. Estrutura do Serviço Container	11
4.2.1. Automação	12
4.2.2. Runtime	13
4.2.3. Relação entre Docker e Kubernetes	14
5. Análise de Soluções Kubernetes	15
5.1. Kubernetes	15
5.2. Comparação entre soluções Kubernetes	15
5.2.1. Kind	15
5.2.2. MicroK8s	16
5.2.3. K3s	17
5.3. Comparação Final das Soluções	17
5.4. Comparação Final das Soluções	17
5.5. Comparação das soluções	18
6. Implementação da Solução Kubernetes	19
6.1. Criação do Cluster Kind	19
6.2. Instalação do Kind	19
6.3. Criação do Cluster Kubernetes	20
6.4. Configuração da API Kubernetes	20
6.5. Tecnologias Utilizadas	21
7. Aplicação para Gestão do Kubernetes	22
7.1. Dashboard	22
7.2. Nodes	22
7.3. Namespaces	23
7.4. Pods	24
7.5. Deployments	24
7.6. Services	25
7.7. Ingress	26
7.8. Experiência de Utilização (UX)	27
8. Análise Crítica e Propostas de Melhoria	29
8.1. Dificuldades Encontradas	29
8.2. Aspectos Positivos do Projeto	29
8.3. Qualidade da Solução Desenvolvida	30
8.4. Melhorias Futuras	31
9. Conclusão	32
10. Bibliografia	33

Lista de Figuras

Figure 1	Dashboard da aplicação Kubernetes	7
Figure 2	Arquitetura a comparar entre containers e Vms	9
Figure 3	Diagrama da estrutura dos containers	11
Figure 4	Docker demonstrando os containers	12
Figure 5	Dockerfile	13
Figure 6	Diagrama funcionamento d kubernetes + docker	14
Figure 7	Diagrama funcionamento d kubernetes + docker	16
Figure 8	Containers no docker desktop	19
Figure 9	Versão do kind	20
Figure 10	Clusters em funcionamento	20
Figure 11	Kubectl Proxy	21
Figure 12	Dashboard da aplicação Kubernetes	22
Figure 13	Kubectl Proxy	23
Figure 14	Namespaces	23
Figure 15	Pods	24
Figure 16		25
Figure 17	Services	26
Figure 18	Ingress	27
Figure 19	Kubectl Proxy	28

Lista de Tabelas

Table 1	Comparação entre Containers e Máquinas Virtuais	8
Table 2	Estrutura de uma imagem Docker	12
Table 3	Componentes do runtime dos containers	14
Table 4	Comparação geral entre soluções Kubernetes	15
Table 5	Vantagens e desvantagens do Kind	16
Table 6	Vantagens e desvantagens do MicroK8s	16
Table 7	Vantagens e desvantagens do K3s	17
Table 8	Comparação final entre as soluções Kubernetes	17
Table 9	Comparação final entre as soluções Kubernetes	17
Table 10	Comparação entre soluções Kubernetes	18
Table 11	Comparação técnica das soluções Kubernetes	18
Table 12	Configuração do cluster Kind	19
Table 13	Estado dos nodes do cluster Kind	20
Table 14	Tecnologias utilizadas no projeto	21
Table 15	Componentes associados ao Ingress	27
Table 16	Principais dificuldades encontradas durante o desenvolvimento	29
Table 17	Possíveis melhorias futuras	31

1. Lista de Siglas e Acrónimos

Sigla	Descrição
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit
Docker	Plataforma de containers
ESTG	Escola Superior de Tecnologia e Gestão
IPL	Instituto Politécnico de Leiria
K8s	Kubernetes
LTI	Laboratório de Tecnologias de Informação
RAM	Random Access Memory
REST	Representational State Transfer
TL2	Trabalho Laboratorial nº2
UC	Unidade Curricular
VM	Virtual Machine

2. Resumo

O presente Trabalho Laboratorial nº2 foi desenvolvido no âmbito da unidade curricular de Laboratório de Tecnologias de Informação do curso de Engenharia Informática da Escola Superior de Tecnologia e Gestão do Instituto Politécnico de Leiria, no ano letivo 2025/2026.

O principal objetivo deste projeto consistiu no desenvolvimento de uma aplicação frontend para gestão de recursos do orquestrador Kubernetes, permitindo a monitorização e administração de clusters através da API disponibilizada pelo Kubernetes.

Numa primeira fase foi realizada uma análise comparativa de diferentes soluções Kubernetes, incluindo Kind, MicroK8s e K3s, avaliando aspetos como facilidade de instalação, consumo de recursos, desempenho, suporte multi-node e adequação ao contexto académico e laboratorial.

Após a análise das várias soluções disponíveis, foi escolhida a plataforma Kind para implementação do cluster Kubernetes. A infraestrutura implementada é composta por um control-plane node e dois worker nodes, permitindo simular um ambiente distribuído semelhante a infraestruturas reais utilizadas em ambientes empresariais.

Posteriormente foi desenvolvida uma aplicação web moderna para interação com a API do Kubernetes. A aplicação permite listar, criar e eliminar recursos fundamentais do cluster, tais como namespaces, pods, deployments e services. Foi ainda implementado um dashboard central com métricas e informações em tempo real relativas ao estado do cluster e dos nodes.

Ao longo do desenvolvimento do projeto foram utilizados conhecimentos relacionados com virtualização, containers, Kubernetes, APIs REST, frontend development e gestão de infraestruturas distribuídas, permitindo consolidar competências importantes na área de Tecnologias de Informação e DevOps.

Palavras-chave: Kubernetes, Docker, Containers, API REST, Kind, DevOps

3. Abstract

This laboratory project was developed within the scope of the Information Technology Laboratory course of the Computer Engineering degree at Escola Superior de Tecnologia e Gestão, Instituto Politécnico de Leiria, during the academic year 2025/2026.

The main objective of this project was the development of a frontend application for Kubernetes resource management, allowing cluster monitoring and administration through the Kubernetes API.

In the first phase, a comparative analysis of several Kubernetes solutions was carried out, including Kind, MicroK8s and K3s, evaluating aspects such as ease of installation, resource consumption, performance, multi-node support and suitability for academic environments.

After analysing the available solutions, Kind was selected for the implementation of the Kubernetes cluster. The implemented infrastructure consists of one master node and two worker nodes, simulating a distributed environment similar to real enterprise infrastructures.

A modern web application was then developed to interact with the Kubernetes API. The application allows listing, creation and deletion of essential cluster resources such as namespaces, pods, deployments and services. A dashboard containing real-time cluster and node metrics was also implemented.

Throughout the project development, knowledge related to virtualization, containers, Kubernetes, REST APIs, frontend development and distributed infrastructure management was applied, allowing the consolidation of important skills in the areas of Information Technology and DevOps.

Keywords: Kubernetes, Docker, Containers, REST API, Kind, DevOps

4. Introdução

No âmbito da unidade curricular de Laboratório de Tecnologias de Informação, foi proposto o desenvolvimento de uma aplicação capaz de interagir com o orquestrador Kubernetes através da sua API, permitindo realizar operações de gestão e monitorização de recursos do cluster.

O Kubernetes é atualmente uma das plataformas de orquestração de containers mais utilizadas no mercado, sendo amplamente adotado em ambientes cloud e infraestruturas distribuídas. A sua utilização permite automatizar processos relacionados com deployment, escalabilidade, monitorização e recuperação de falhas em aplicações baseadas em containers.

O presente projeto teve como principal objetivo analisar diferentes soluções Kubernetes, implementar uma infraestrutura distribuída com múltiplos nodes e desenvolver uma aplicação frontend moderna para gestão dos recursos do cluster.

Numa fase inicial foi realizada uma análise comparativa entre várias distribuições Kubernetes, avaliando vantagens, desvantagens e adequação de cada solução ao contexto do trabalho laboratorial. Posteriormente foi implementado um cluster Kubernetes utilizando a solução Kind, composto por um control-plane node e dois worker nodes.

Após a implementação da infraestrutura, foi desenvolvida uma aplicação web responsável pela interação com a API do Kubernetes. A aplicação disponibiliza funcionalidades de gestão de namespaces, pods, deployments e services, assim como um dashboard central com informações relativas ao estado do cluster e utilização de recursos.

Este projeto permitiu aprofundar conhecimentos nas áreas de virtualização, containers, Kubernetes, APIs REST, frontend development e DevOps, contribuindo significativamente para a consolidação de competências técnicas relevantes no contexto atual das infraestruturas distribuídas.



Figure 1: Dashboard da aplicação Kubernetes

4.1. Containers vs Virtual Machines

Antes da implementação de qualquer solução Kubernetes, é importante compreender a diferença entre containers e máquinas virtuais, uma vez que ambas as tecnologias são utilizadas em ambientes de virtualização, embora possuam arquiteturas e objetivos diferentes.

As máquinas virtuais (Virtual Machines – VM's) permitem executar vários sistemas operativos independentes sobre o mesmo hardware físico através da utilização de um hypervisor. Cada máquina

virtual possui o seu próprio sistema operativo, kernel, drivers e aplicações, funcionando de forma completamente isolada das restantes.

Por outro lado, os containers utilizam virtualização ao nível do sistema operativo, partilhando o mesmo kernel do host. Em vez de incluir um sistema operativo completo, os containers incluem apenas as dependências necessárias para execução da aplicação.

Esta diferença permite que os containers sejam significativamente mais leves, rápidos e eficientes em comparação com as máquinas virtuais tradicionais.

Enquanto uma máquina virtual pode demorar vários minutos a iniciar, um container consegue arrancar em poucos segundos.

Os containers apresentam também vantagens ao nível da escalabilidade, portabilidade e utilização eficiente de recursos computacionais.

No entanto, as máquinas virtuais continuam a oferecer um maior nível de isolamento e segurança, sendo ainda bastante utilizadas em ambientes críticos ou em aplicações que necessitam de sistemas operativos distintos.

Característica	Containers	Máquinas Virtuais
Virtualização	Sistema Operativo	Hardware
Kernel	Partilhado	Individual
Consumo de Recursos	Baixo	Elevado
Velocidade de Arranque	Muito rápida	Mais lenta
Portabilidade	Elevada	Média
Escalabilidade	Alta	Média
Isolamento	Médio	Elevado
Desempenho	Muito elevado	Elevado
Utilização Típica	Microserviços	Infraestruturas tradicionais

Table 1: Comparação entre Containers e Máquinas Virtuais

Os containers tornaram-se atualmente uma das principais tecnologias utilizadas em ambientes cloud-native, DevOps e infraestruturas distribuídas, sendo amplamente utilizados em plataformas modernas como Docker e Kubernetes.

A utilização de containers permite às organizações implementar aplicações de forma mais rápida, eficiente e escalável, reduzindo significativamente os custos de infraestrutura e aumentando a flexibilidade das aplicações.

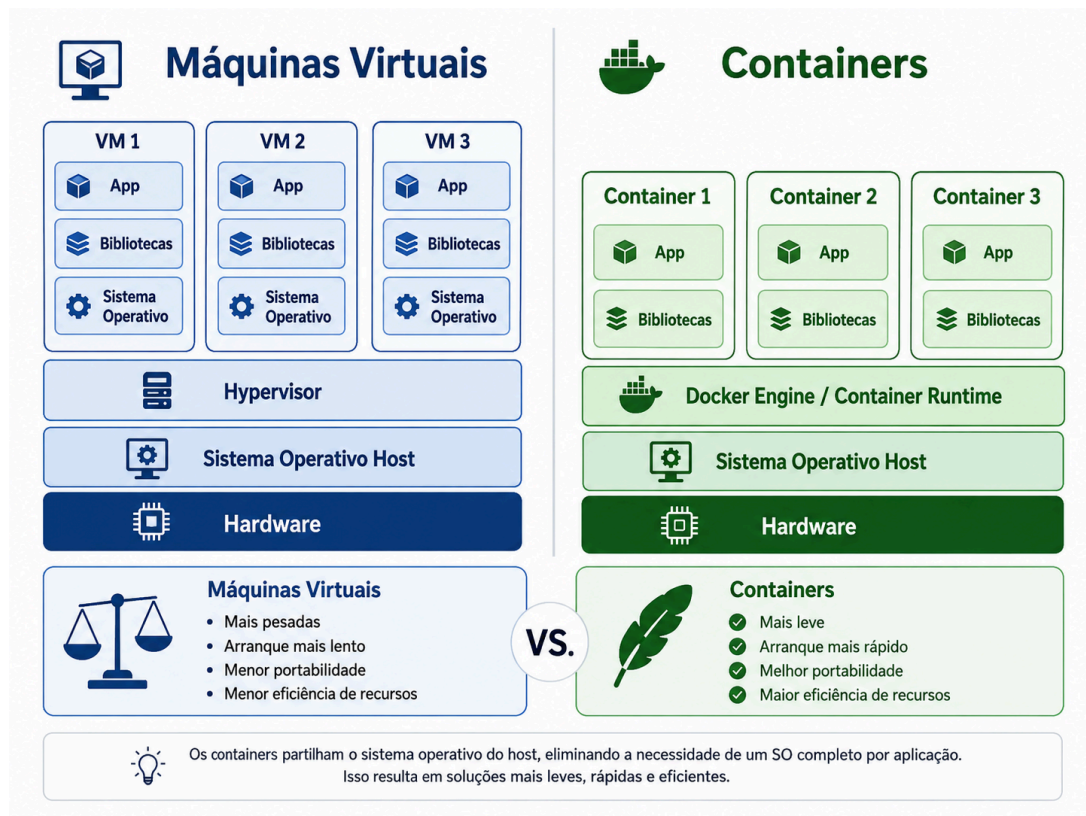


Figure 2: Arquitetura a comparar entre containers e Vms

4.1.1. Vantagens da Utilização de Containers

A utilização de containers apresenta diversas vantagens importantes no desenvolvimento e implementação de aplicações modernas.

Entre as principais vantagens destacam-se:

- Arranque extremamente rápido
- Baixo consumo de recursos
- Elevada portabilidade
- Facilidade de escalabilidade
- Isolamento entre aplicações
- Facilidade de distribuição
- Maior eficiência computacional
- Melhor integração com arquiteturas cloud-native

Estas características tornam os containers extremamente adequados para ambientes distribuídos e aplicações baseadas em microserviços.

4.1.2. Desvantagens da Utilização de Containers

Apesar das suas vantagens, os containers apresentam também algumas limitações.

As principais desvantagens são:

- Menor isolamento comparativamente a VM's
- Dependência do kernel do host
- Algumas limitações de segurança
- Maior complexidade em ambientes distribuídos grandes
- Necessidade de ferramentas de orquestração

Para resolver muitos destes problemas surgiram plataformas de orquestração como o Kubernetes.

4.2. Estrutura do Serviço Container

Para compreender corretamente o funcionamento do Kubernetes é também importante perceber como funciona internamente uma infraestrutura baseada em containers.

Neste projeto foi utilizado o Docker como principal tecnologia de containers, sendo atualmente uma das plataformas mais utilizadas mundialmente para criação, distribuição e execução de containers.

O Docker permite encapsular aplicações juntamente com todas as suas dependências, garantindo que estas podem ser executadas de forma consistente em diferentes ambientes.

O funcionamento de uma infraestrutura baseada em containers pode ser dividido em três componentes principais:

- Imagens
- Automação
- Runtime

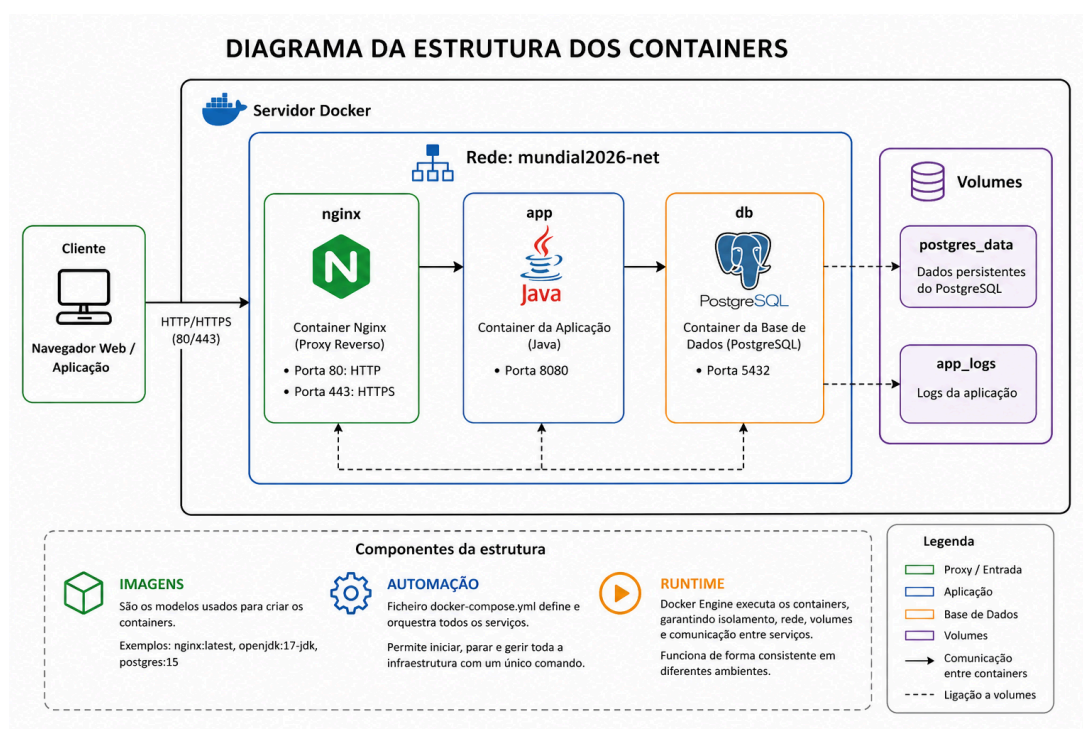


Figure 3: Diagrama da estrutura dos containers

Uma imagem contém todos os componentes necessários para execução de uma aplicação, incluindo:

- Sistema base
- Bibliotecas
- Dependências
- Aplicações
- Configurações

As imagens funcionam de forma hierárquica, sendo possível criar novas imagens a partir de imagens já existentes.

Por exemplo, uma imagem Ubuntu pode servir de base para uma imagem Nginx, que por sua vez pode servir de base para uma aplicação web personalizada.

As imagens Docker são normalmente armazenadas em registries, sendo o Docker Hub uma das plataformas mais utilizadas.

Exemplo de download de uma imagem Docker:

```
docker pull nginx
```

Exemplo de execução de um container:

```
docker run -d -p 80:80 nginx
```

Uma das principais vantagens das imagens Docker é a sua elevada portabilidade, permitindo executar a mesma aplicação em diferentes sistemas operativos e infraestruturas sem necessidade de alterações.

Componente	Função
Imagem Base	Sistema operativo base
Dependências	Bibliotecas necessárias
Aplicação	Software executado
Configurações	Parâmetros do container
Volumes	Persistência de dados

Table 2: Estrutura de uma imagem Docker

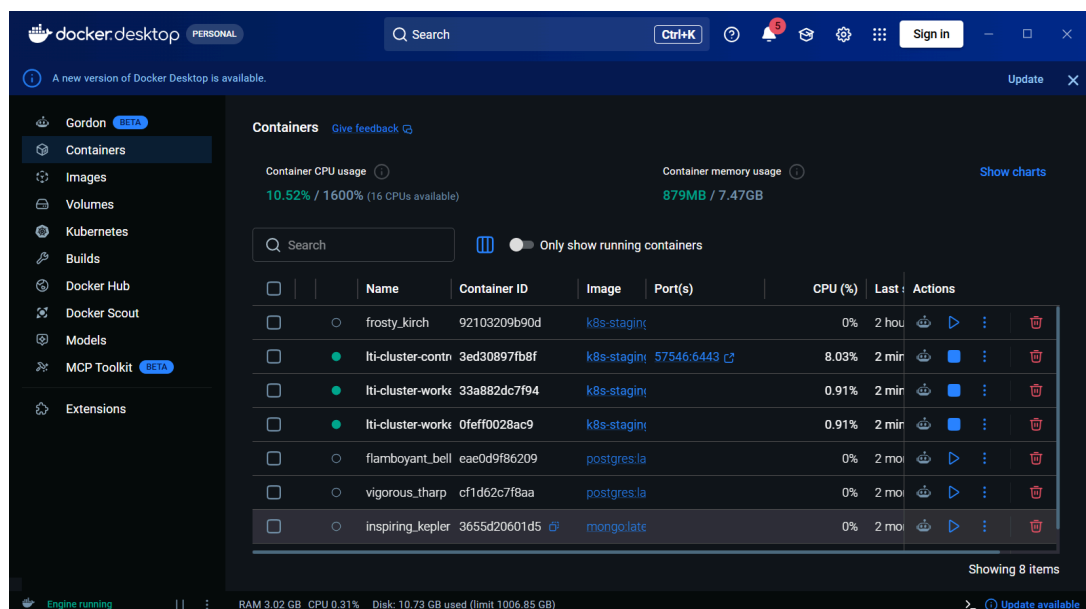


Figure 4: Docker demonstrando os containers

4.2.1. Automação

Outro componente fundamental na infraestrutura de containers é a automação.

A automação permite criar, configurar, atualizar e remover containers de forma dinâmica e automatizada.

No Docker, esta automação é normalmente realizada através de Dockerfiles.

Um Dockerfile consiste num ficheiro de texto contendo instruções responsáveis pela construção automática de imagens Docker.

Exemplo de Dockerfile:

```
FROM nginx:latest

COPY . /usr/share/nginx/html
```

EXPOSE 80

A utilização de automação permite:

- Reprodutibilidade
- Rapidez de deployment
- Redução de erros humanos
- Maior consistência entre ambientes
- Facilidade de manutenção

Estas características são extremamente importantes em ambientes DevOps e CI/CD.

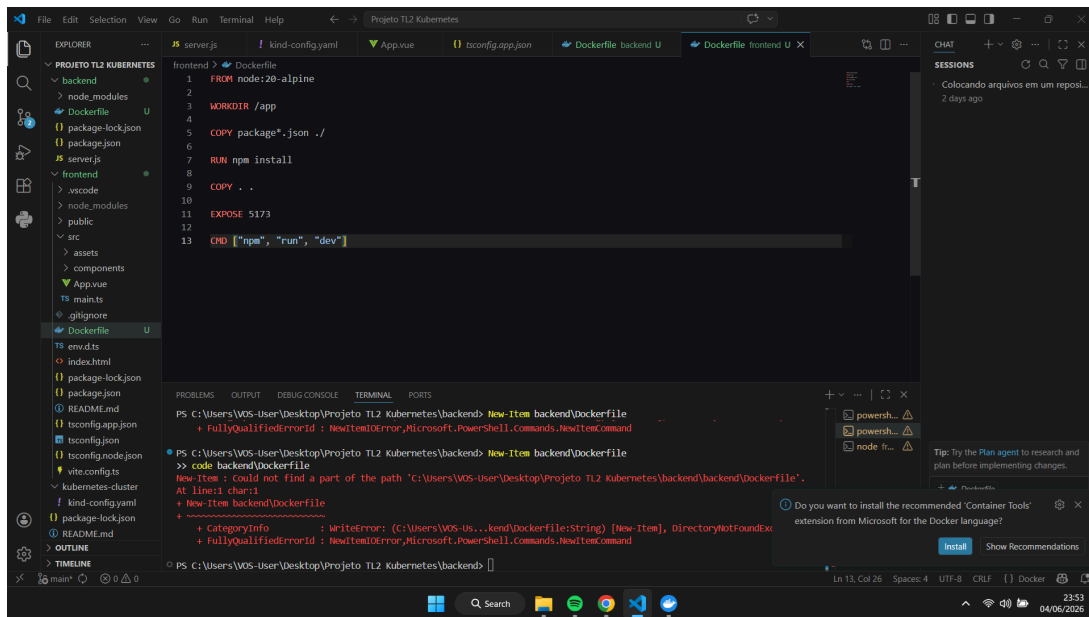


Figure 5: Dockerfile

4.2.2. Runtime

O runtime corresponde à camada responsável pela execução efetiva dos containers.

Quando um container é iniciado, este passa a executar como um processo isolado dentro do sistema operativo anfitrião.

Os containers executam de forma isolada através da utilização de namespaces e cgroups do Linux, permitindo limitar recursos como:

- CPU
- Memória RAM
- Rede
- Processos
- Armazenamento

Uma das grandes vantagens do runtime dos containers é a rapidez com que os containers conseguem ser iniciados ou destruídos.

Isto torna os containers extremamente adequados para ambientes escaláveis e aplicações distribuídas.

O runtime permite ainda:

- Escalabilidade horizontal

- Balanceamento de carga
- Recuperação automática de falhas
- Distribuição dinâmica de workloads

Estas funcionalidades tornam os containers ideais para utilização com plataformas de orquestração como Kubernetes.

Funcionalidade	Objetivo
Namespaces	Isolamento entre containers
Cgroups	Limitação de recursos
Runtime	Execução dos containers
Rede	Comunicação entre containers
Volumes	Persistência de dados

Table 3: Componentes do runtime dos containers

4.2.3. Relação entre Docker e Kubernetes

Embora o Docker seja responsável pela criação e execução dos containers, o Kubernetes é responsável pela orquestração desses containers.

O Kubernetes permite gerir automaticamente:

- Deployments
- Escalabilidade
- Balanceamento de carga
- Recuperação de falhas
- Monitorização
- Distribuição de workloads

Desta forma, o Docker e o Kubernetes funcionam de forma complementar, sendo atualmente uma das arquiteturas mais utilizadas em ambientes cloud-native.

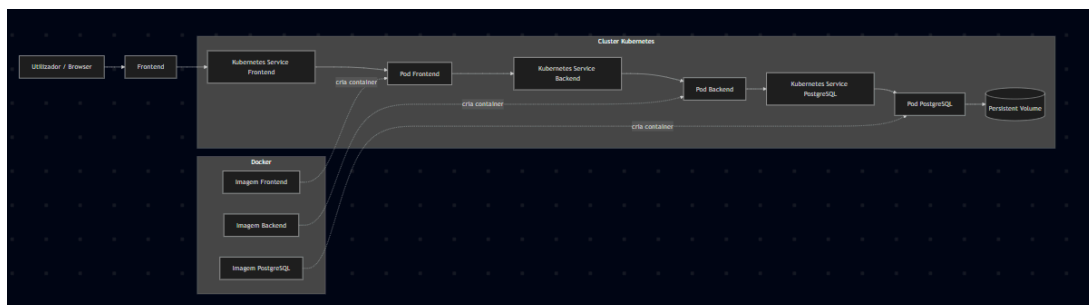


Figure 6: Diagrama funcionamento d kubernetes + docker

5. Análise de Soluções Kubernetes

5.1. Kubernetes

O Kubernetes é uma plataforma open-source de orquestração de containers desenvolvida inicialmente pela Google e atualmente mantida pela Cloud Native Computing Foundation (CNCF).

A principal função do Kubernetes consiste na automatização da gestão de aplicações distribuídas em containers, permitindo funcionalidades como deployment automático, balanceamento de carga, escalabilidade, recuperação de falhas e monitorização de recursos.

5.2. Comparação entre soluções Kubernetes

Durante a fase inicial do projeto foram analisadas várias soluções Kubernetes, nomeadamente o Kind, MicroK8s e K3s. A análise teve como principal objetivo identificar qual a solução mais adequada para o contexto do trabalho laboratorial, tendo em consideração fatores como facilidade de instalação, consumo de recursos, desempenho, suporte multi-node e simplicidade de configuração.

Cada solução apresenta características específicas e diferentes objetivos de utilização. Algumas soluções são mais direcionadas para ambientes de desenvolvimento local, enquanto outras se focam em ambientes edge, IoT ou pipelines CI/CD.

Solução	Tipo	Objetivo	Complexidade	Consumo
Kind	Docker Containers	CI/CD e Testes	Baixa	Médio
MicroK8s	Distribuição Kubernetes	Laboratorial	Baixa	Baixo
K3s	Distribuição Lightweight	IoT e Edge	Baixa	Muito Baixo

Table 4: Comparação geral entre soluções Kubernetes

5.2.1. Kind

O Kind, cujo nome significa Kubernetes IN Docker, é uma solução Kubernetes que utiliza containers Docker para simular nodes Kubernetes.

O Kind não depende diretamente de máquinas virtuais tradicionais, utilizando containers Docker para criar os nodes do cluster.

Exemplo de criação de cluster:

```
kind create cluster
```

Exemplo de configuração multi-node:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
```

```
nodes:
```

- role: control-plane
- role: worker
- role: worker

Vantagens	Desvantagens
• Inicialização extremamente rápida • Excelente integração com Docker • Muito utilizado em CI/CD • Fácil criação de clusters multi-node • Configuração simplificada	• Dependência obrigatória do Docker • Menor aproximação a ambientes reais • Limitações em redes complexas • Menos intuitivo para iniciantes

Table 5: Vantagens e desvantagens do Kind

O Kind foi a solução escolhida para este projeto, por permitir criar um cluster Kubernetes local através de containers Docker, com configuração simples, arranque rápido e suporte para topologias multi-node.

```
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
lti-cluster-control-plane           Ready    control-plane   2d6h   v1.36.1
lti-cluster-worker                  Ready    <none>         2d6h   v1.36.1
lti-cluster-worker2                 Ready    <none>         2d6h   v1.36.1
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend>
```

Figure 7: Diagrama funcionamento d kubernetes + docker

5.2.2. MicroK8s

O MicroK8s é uma distribuição Kubernetes leve desenvolvida pela Canonical, empresa responsável pelo Ubuntu.

Esta solução foi criada com o objetivo de disponibilizar um ambiente Kubernetes simples, rápido e eficiente, sendo especialmente adequada para ambientes laboratoriais, desenvolvimento local e aprendizagem.

Exemplo de instalação:

```
snap install microk8s --classic
```

Exemplo de ativação de addons:

```
microk8s enable dns
microk8s enable dashboard
microk8s enable metrics-server
```

O MicroK8s suporta ainda clusters multi-node, alta disponibilidade e integração simples com ferramentas Kubernetes tradicionais.

Vantagens	Desvantagens
• Muito leve e eficiente • Baixo consumo de recursos • Instalação extremamente simples • Excelente integração com Ubuntu • Suporte multi-node • Addons integrados • Aproximação próxima de ambientes reais	• Dependência do Snap • Algumas configurações avançadas manuais • Menor foco em ambientes enterprise complexos

Table 6: Vantagens e desvantagens do MicroK8s

Apesar das vantagens apresentadas, o MicroK8s não foi utilizado na implementação final, uma vez que o projeto foi desenvolvido com Kind sobre Docker Desktop.

5.2.3. K3s

O K3s é uma distribuição Kubernetes lightweight desenvolvida pela Rancher, tendo sido otimizada para ambientes edge computing, IoT e dispositivos com poucos recursos.

Esta solução foi desenvolvida com o objetivo de reduzir significativamente o consumo de recursos e simplificar a instalação e manutenção do Kubernetes.

Exemplo de instalação:

```
curl -sL https://get.k3s.io | sh -
```

O K3s consegue funcionar corretamente em dispositivos com hardware limitado, como Raspberry Pi ou pequenos servidores edge.

Vantagens	Desvantagens
- Consumo extremamente reduzido - Muito rápido e leve - Excelente para IoT e Edge Computing - Instalação simplificada - Boa performance em hardware limitado	- Menos adequado para ambientes tradicionais - Algumas funcionalidades removidas - Menor utilização empresarial - Menor integração com alguns addons

Table 7: Vantagens e desvantagens do K3s

Apesar das suas vantagens em ambientes edge e IoT, o K3s não foi escolhido devido ao facto de o objetivo principal do projeto se focar mais em aprendizagem, administração e gestão tradicional de clusters Kubernetes.

5.3. Comparação Final das Soluções

Após a análise das várias soluções Kubernetes, concluiu-se que todas apresentam vantagens específicas dependendo do tipo de utilização pretendida.

Solução	Facilidade	Performance	Escalabilidade	Consumo	Adequação ao Projeto
Kind	Muito Alta	Muito Boa	Boa	Médio	Boa
MicroK8s	Muito Alta	Muito Boa	Alta	Baixo	Excelente
K3s	Alta	Muito Boa	Alta	Muito Baixo	Boa

Table 8: Comparação final entre as soluções Kubernetes

Com base nos testes realizados e nas características analisadas, a solução Kind foi considerada a mais adequada para o desenvolvimento do presente Trabalho Laboratorial, principalmente devido à sua simplicidade de utilização, baixo consumo de recursos, facilidade de configuração e excelente compatibilidade com ambientes académicos e laboratoriais.

5.4. Comparação Final das Soluções

Após a análise das várias soluções Kubernetes, concluiu-se que todas apresentam vantagens específicas dependendo do tipo de utilização pretendida.

Solução	Facilidade	Performance	Escalabilidade	Consumo	Adequação ao Projeto
Kind	Muito Alta	Muito Boa	Boa	Médio	Boa
MicroK8s	Muito Alta	Muito Boa	Alta	Baixo	Excelente
K3s	Alta	Muito Boa	Alta	Muito Baixo	Boa

Table 9: Comparação final entre as soluções Kubernetes

Com base nos testes realizados e nas características analisadas, a solução Kind foi considerada a mais adequada para o desenvolvimento do presente Trabalho Laboratorial, principalmente devido à sua simplicidade de utilização, baixo consumo de recursos, facilidade de configuração e excelente compatibilidade com ambientes acadêmicos e laboratoriais.

5.5. Comparação das soluções

Solução	Consumo	Facilidade	Multi-node	Adequação
Kind	Médio	Alta	Sim	CI/CD
MicroK8s	Baixo	Muito Alta	Sim	Laboratorial
K3s	Muito Baixo	Alta	Sim	IoT/Edge

Table 10: Comparação entre soluções Kubernetes

Solução	Instalação	Performance	Escalabilidade	Recursos
Kind	Muito Fácil	Boa	Boa	Médios
MicroK8s	Muito Fácil	Muito Boa	Alta	Baixos
K3s	Fácil	Muito Boa	Alta	Muito Baixos

Table 11: Comparação técnica das soluções Kubernetes

Após a análise realizada, foi escolhida a solução Kind devido à sua simplicidade de instalação, suporte multi-node, reduzido consumo de recursos e facilidade de integração com a API Kubernetes.

6. Implementação da Solução Kubernetes

6.1. Criação do Cluster Kind

Para a implementação da infraestrutura Kubernetes foi utilizado o Kind, que permite criar clusters Kubernetes locais usando containers Docker como nodes.

A infraestrutura foi configurada com:

- 1 node control-plane
- 2 worker nodes

Esta configuração permite simular um cluster distribuído em ambiente local, cumprindo o requisito de possuir um nó de controlo e dois nós de trabalho.

Node	Função	Tecnologia
Control Plane	Gestão do cluster	Container Docker
Worker 1	Execução de Pods	Container Docker
Worker 2	Execução de Pods	Container Docker

Table 12: Configuração do cluster Kind

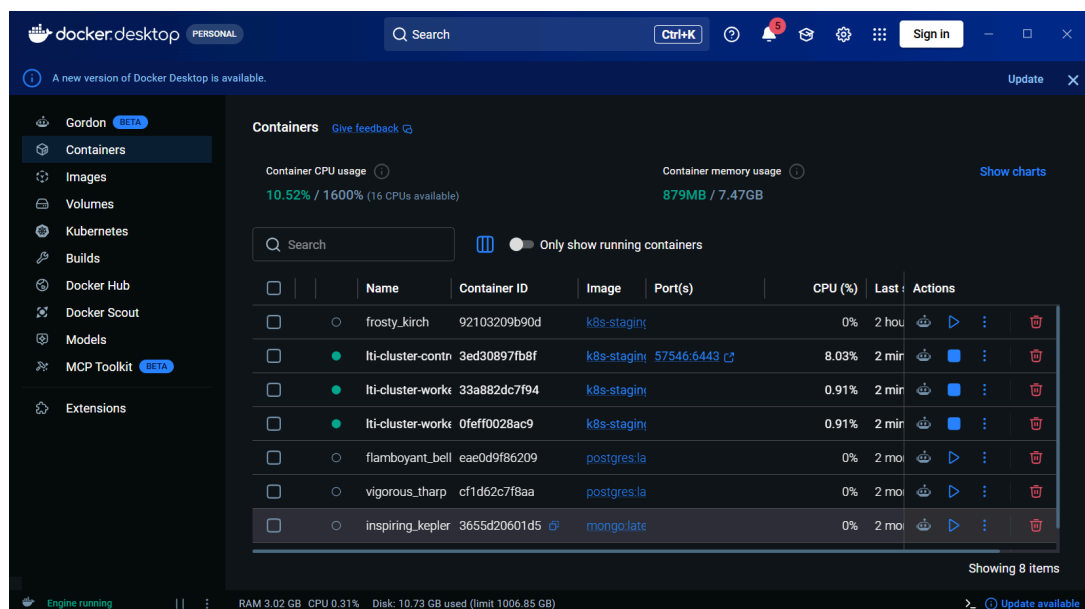


Figure 8: Containers no docker desktop

6.2. Instalação do Kind

A instalação do Kind foi realizada no sistema anfitrião, recorrendo ao gestor de pacotes do Windows.

```
winget install Kubernetes.kind
```

Após a instalação, foi possível confirmar a versão instalada com o seguinte comando:

```
kind version
```

O Kind utiliza o Docker para criar os nodes Kubernetes como containers, permitindo uma configuração rápida e adequada para testes locais e ambientes laboratoriais.

```
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
liti-cluster-control-plane         Ready    control-plane   2d6h   v1.36.1
liti-cluster-worker               Ready    <none>        2d6h   v1.36.1
liti-cluster-worker2              Ready    <none>        2d6h   v1.36.1
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend>
```

Figure 9: Versão do kind

6.3. Criação do Cluster Kubernetes

A criação do cluster foi realizada através de um ficheiro de configuração do Kind, definindo um node control-plane e dois worker nodes.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
```

Depois de criado o ficheiro de configuração, o cluster foi iniciado com o seguinte comando:

```
kind create cluster --name liti-cluster --config kind-config.yaml
```

Após a criação do cluster, foi possível verificar o estado dos nodes através do comando:

```
kubectl get nodes
```

Como resultado, o cluster apresentou três nodes:

- 1 Control Plane Node
- 2 Worker Nodes

Node	Função	Estado
liti-cluster-control-plane	Control Plane	Ready
liti-cluster-worker	Worker	Ready
liti-cluster-worker2	Worker	Ready

Table 13: Estado dos nodes do cluster Kind

```
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
liti-cluster-control-plane         Ready    control-plane   2d6h   v1.36.1
liti-cluster-worker               Ready    <none>        2d6h   v1.36.1
liti-cluster-worker2              Ready    <none>        2d6h   v1.36.1
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend>
```

Figure 10: Clusters em funcionamento

6.4. Configuração da API Kubernetes

Para permitir a comunicação entre a aplicação frontend e o cluster Kubernetes foi necessário disponibilizar externamente a API do Kubernetes.

Para isso foi executado o seguinte comando no master node:

```
kubectl proxy --address=0.0.0.0 --accept-hosts='^*$'
```

Este comando permite expor a API Kubernetes para acesso externo através da rede local.

Após a configuração, a API ficou acessível através do seguinte formato:

http://localhost:8001

```
liti-cluster-control-plane Ready control-plane 2d6h v1.36.1
liti-cluster-worker Ready <none> 2d6h v1.36.1
liti-cluster-worker2 Ready <none> 2d6h v1.36.1
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend>
PS C:\Users\VOS-User\Desktop\Projeto TL2 Kubernetes\backend> kubectl proxy
Starting to serve on 127.0.0.1:8001
```

Figure 11: Kubectl Proxy

6.5. Tecnologias Utilizadas

Durante o desenvolvimento da aplicação foram utilizadas várias tecnologias relacionadas com virtualização, frontend development e gestão de containers.

Tecnologia	Objetivo
Vue.js	Frontend da aplicação
Node.js	Backend/API
Express	Servidor backend
Axios	Pedidos HTTP
Kind	Cluster Kubernetes local
Docker	Containers
Docker Desktop	Execução dos nodes Kind
Windows	Sistema operativo anfitrião

Table 14: Tecnologias utilizadas no projeto

7. Aplicação para Gestão do Kubernetes

7.1. Dashboard

A página principal da aplicação consiste num dashboard responsável pela apresentação das principais informações relativas ao cluster Kubernetes.

No dashboard são apresentadas métricas em tempo real sobre:

- Utilização de CPU
- Utilização de memória RAM
- Estado dos nodes
- Número de pods ativos
- Deployments existentes
- Services existentes

Esta secção permite ao utilizador obter rapidamente uma visão geral do estado do cluster Kubernetes.

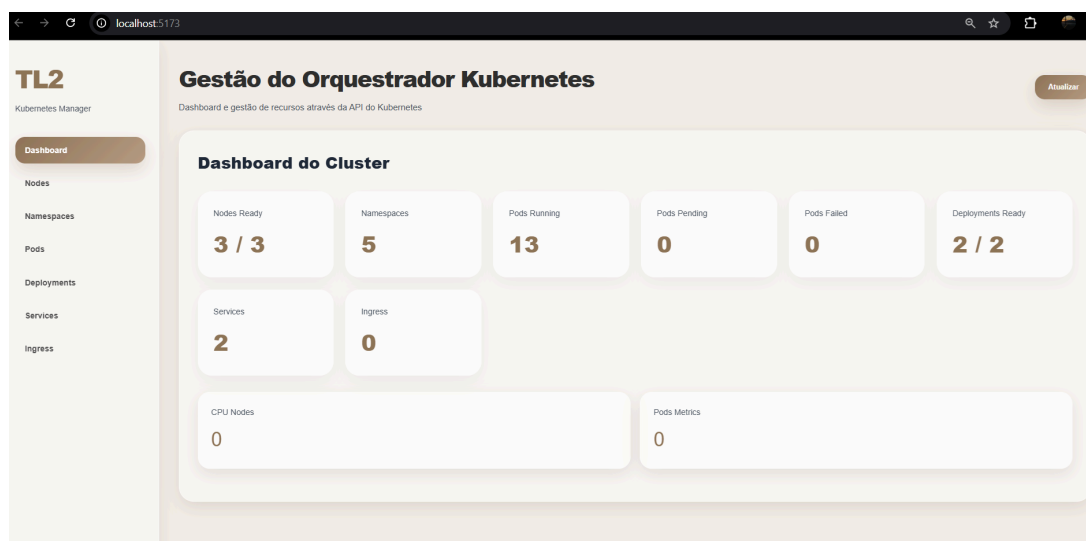


Figure 12: Dashboard da aplicação Kubernetes

7.2. Nodes

A secção de nodes permite listar todos os nodes pertencentes ao cluster Kubernetes.

Para a obtenção dos dados foi realizado um pedido GET ao seguinte endpoint da API Kubernetes:

GET /api/v1/nodes

A listagem apresenta informações relativas ao nome do node, estado, função e versão do Kubernetes.

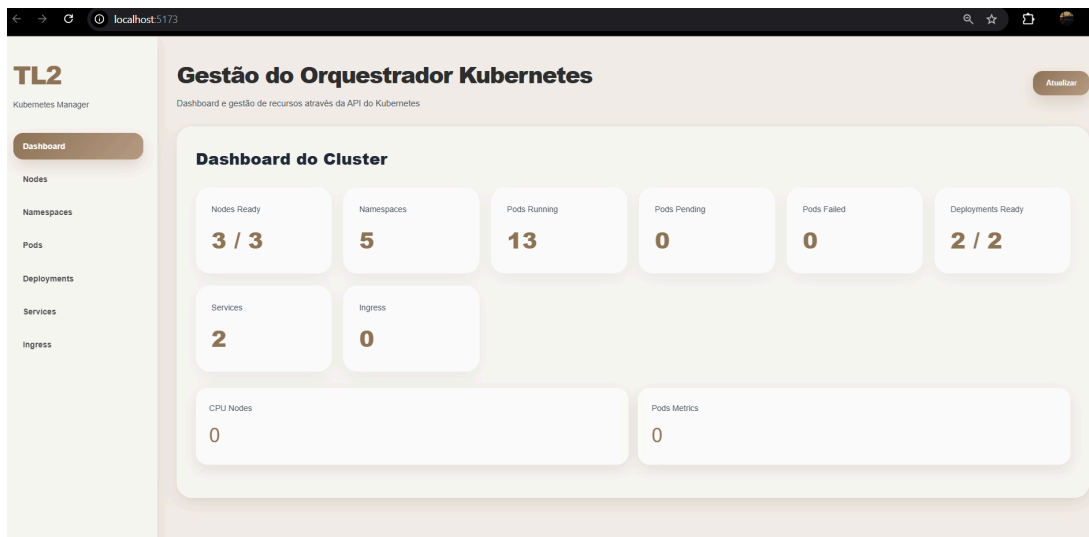


Figure 13: Kubectl Proxy

7.3. Namespaces

A secção de namespaces permite gerir os diferentes espaços lógicos existentes no cluster Kubernetes.

As funcionalidades implementadas foram:

- Listar namespaces
- Criar namespaces
- Eliminar namespaces

Os endpoints utilizados foram os seguintes:

GET /api/v1/namespaces

POST /api/v1/namespaces

DELETE /api/v1/namespaces/{namespace}

O utilizador pode criar novos namespaces através da interface gráfica da aplicação e eliminá-los diretamente através dos botões de ação.

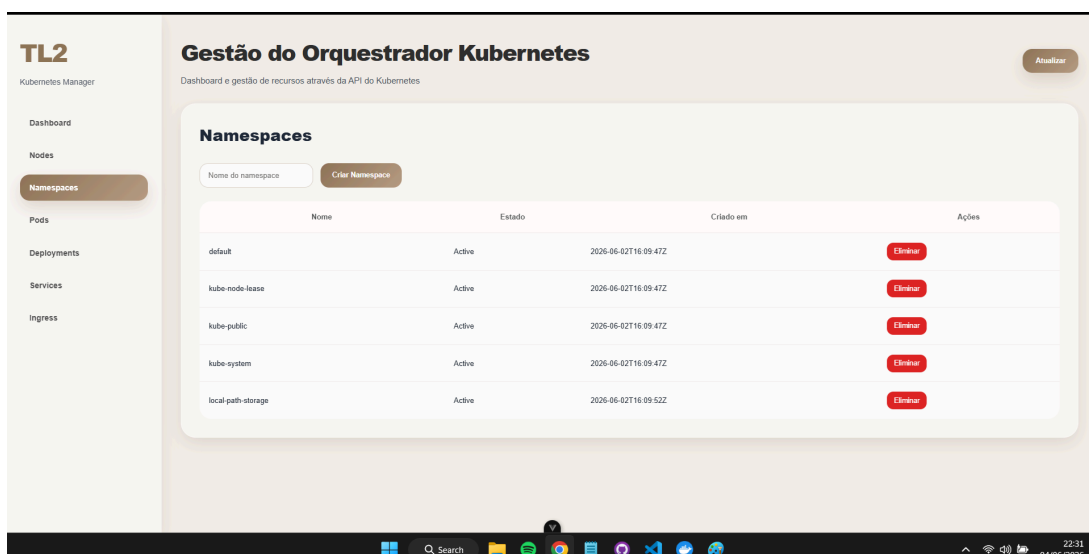


Figure 14: Namespaces

7.4. Pods

Os pods representam a menor unidade de computação no Kubernetes, sendo responsáveis pela execução dos containers.

Na aplicação foram implementadas as seguintes funcionalidades:

- Listar pods
- Criar pods
- Eliminar pods

Endpoints utilizados:

```
GET /api/v1/namespaces/{namespace}/pods
POST /api/v1/namespaces/{namespace}/pods
DELETE /api/v1/namespaces/{namespace}/pods/{pod}
```

Na criação de pods o utilizador pode definir:

- Nome do pod
- Nome do container
- Imagem Docker
- Namespace

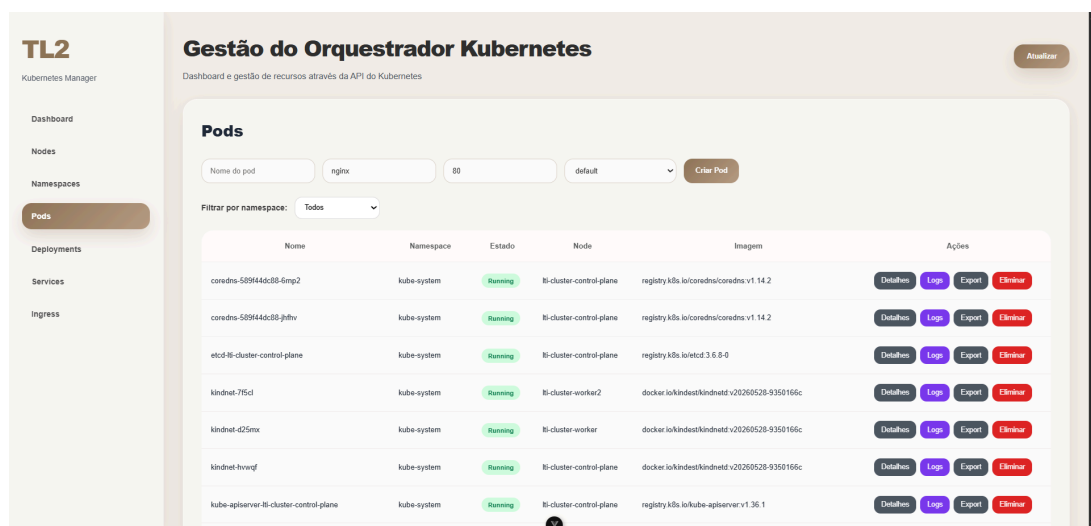


Figure 15: Pods

7.5. Deployments

Os deployments permitem gerir aplicações distribuídas no cluster Kubernetes.

Foram implementadas funcionalidades para:

- Listar deployments
- Criar deployments
- Eliminar deployments

Endpoints utilizados:

```
GET /apis/apps/v1/namespaces/{namespace}/deployments
POST /apis/apps/v1/namespaces/{namespace}/deployments
DELETE /apis/apps/v1/namespaces/{namespace}/deployments/{deployment}
```

Durante a criação de deployments é possível definir:

- Nome do deployment

- Número de réplicas
- Container
- Imagem Docker
- Namespace

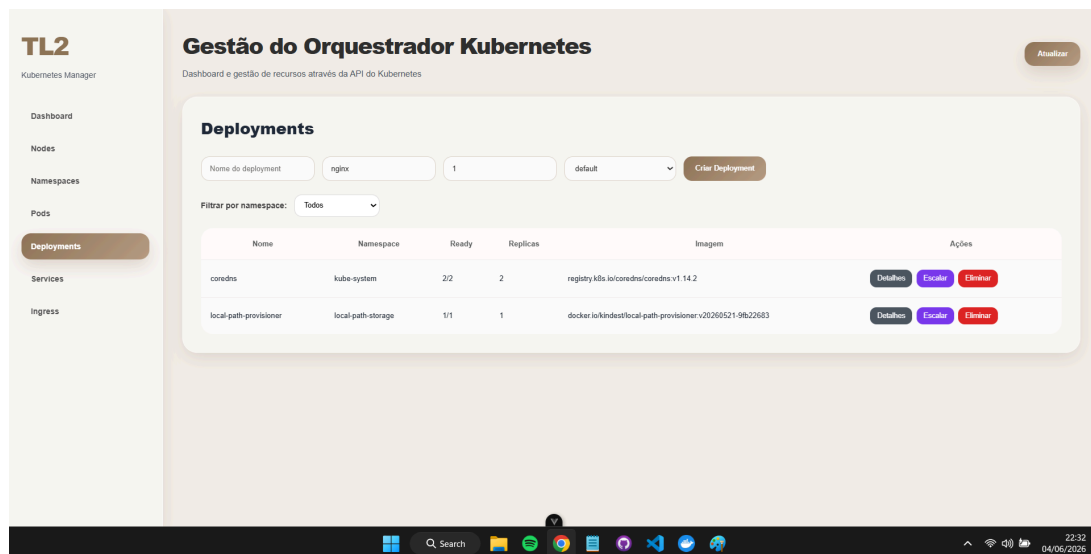


Figure 16:

7.6. Services

Os services permitem expor aplicações executadas no cluster Kubernetes.

Na aplicação foram implementadas funcionalidades para:

- Listar services
- Criar services
- Eliminar services

Endpoints utilizados:

```
GET /api/v1/namespaces/{namespace}/services
POST /api/v1/namespaces/{namespace}/services
DELETE /api/v1/namespaces/{namespace}/services/{service}
```

Na criação de services é possível configurar:

- Nome do service
- Porta
- Target Port
- Tipo de service
- Namespace

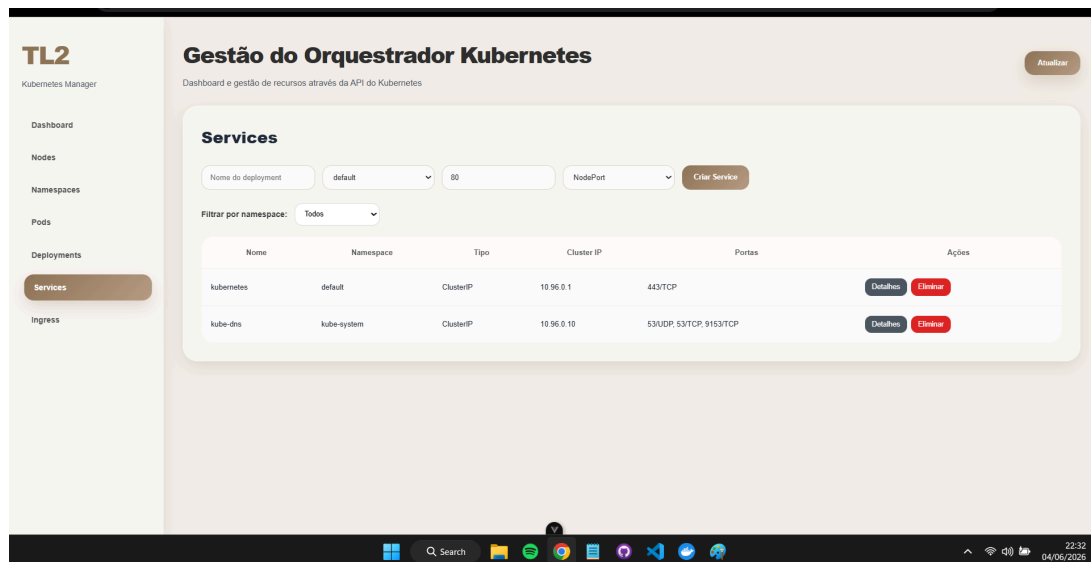


Figure 17: Services

7.7. Ingress

O Ingress é um recurso do Kubernetes responsável por gerir o acesso externo às aplicações que estão a ser executadas dentro do cluster. Enquanto os Services permitem expor pods internamente ou através de portas específicas, o Ingress permite definir regras de encaminhamento HTTP e HTTPS para diferentes serviços da aplicação.

Neste projeto, o Ingress pode ser utilizado para facilitar o acesso à aplicação frontend e ao backend, evitando a necessidade de expor diretamente cada serviço através de portas individuais. Desta forma, é possível centralizar o tráfego externo e encaminhá-lo para o serviço correto dentro do cluster Kubernetes.

O Ingress funciona em conjunto com um Ingress Controller, que é o componente responsável por interpretar as regras definidas e encaminhar os pedidos para os serviços correspondentes. Um dos controladores mais utilizados é o NGINX Ingress Controller.

As principais vantagens da utilização de Ingress são:

Centralização do acesso externo às aplicações; Encaminhamento de pedidos por domínio ou caminho; **Possibilidade de utilização de HTTPS;** Redução da necessidade de expor múltiplas portas; Maior organização na gestão do tráfego externo.

Exemplo de um ficheiro Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
  - host: lti.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
```

```

    port:
      number: 80
- path: /api
  pathType: Prefix
  backend:
    service:
      name: backend-service
      port:
        number: 3000

```

Neste exemplo, os pedidos feitos para o domínio `lti.local` são encaminhados para diferentes serviços consoante o caminho utilizado. Os pedidos para `/` são enviados para o serviço do frontend, enquanto os pedidos para `/api` são encaminhados para o serviço do backend.

No contexto do projeto, o Ingress representa uma melhoria importante para a organização da exposição da aplicação, permitindo uma configuração mais próxima de ambientes reais de produção.

Componente	Função
Ingress	Define regras de acesso externo HTTP/HTTPS
Ingress Controller	Aplica as regras e encaminha o tráfego
Service	Encaminha o tráfego para os pods
Pod	Executa os containers da aplicação

Table 15: Componentes associados ao Ingress

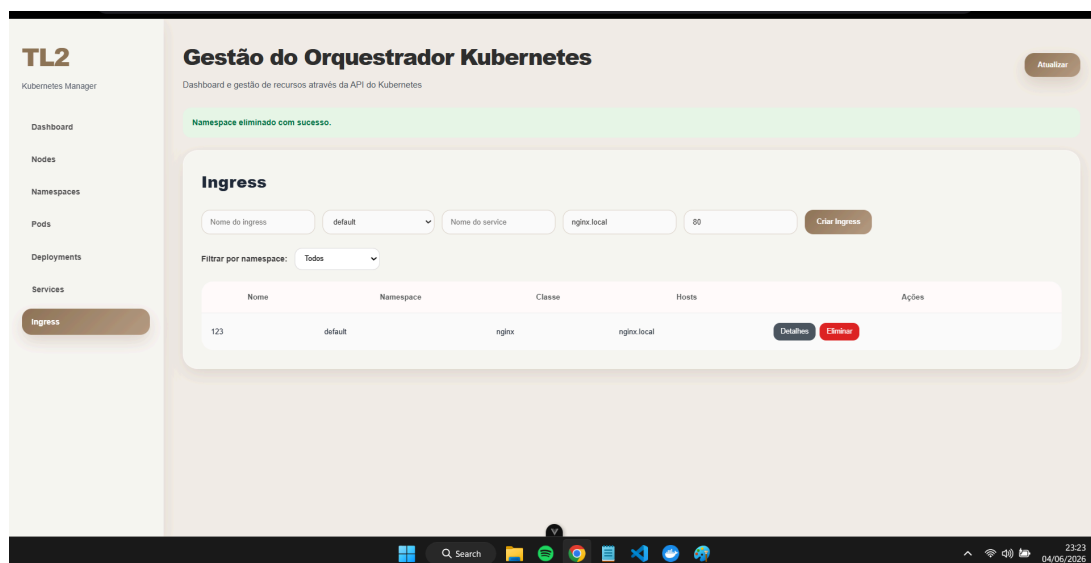


Figure 18: Ingress

7.8. Experiência de Utilização (UX)

Durante o desenvolvimento da aplicação foi dada especial atenção à experiência de utilização.

A interface foi desenvolvida com um design moderno e intuitivo, utilizando tons neutros e menus organizados de forma a facilitar a navegação entre as várias funcionalidades da aplicação.

Foram ainda implementados:

- Feedback visual de sucesso e erro
- Atualização dinâmica de dados
- Navegação simplificada
- Organização por secções

- Interface responsiva



Figure 19: Kubectl Proxy

8. Análise Crítica e Propostas de Melhoria

A realização do presente Trabalho Laboratorial revelou-se bastante importante para a consolidação de conhecimentos relacionados com virtualização, containers, Kubernetes, DevOps e desenvolvimento de aplicações frontend para gestão de infraestruturas distribuídas.

Ao longo do desenvolvimento do projeto foi possível adquirir experiência prática na instalação e configuração de clusters Kubernetes, bem como compreender o funcionamento interno de diversos componentes fundamentais da arquitetura Kubernetes, tais como nodes, pods, deployments, namespaces e services.

Para além disso, o projeto permitiu aprofundar conhecimentos sobre comunicação com APIs REST, integração entre frontend e backend e gestão de aplicações distribuídas em ambientes virtualizados.

Apesar do resultado final ter sido bastante positivo, durante o desenvolvimento surgiram algumas dificuldades técnicas e desafios relacionados com diferentes componentes da infraestrutura e da aplicação desenvolvida.

8.1. Dificuldades Encontradas

Durante a implementação do projeto foram identificadas várias dificuldades técnicas, principalmente relacionadas com a configuração da infraestrutura Kubernetes e comunicação entre os diferentes componentes da aplicação.

As principais dificuldades encontradas foram:

- Configuração inicial do cluster Kubernetes
- Associação correta dos worker nodes ao master node
- Comunicação entre a aplicação frontend e a API Kubernetes
- Exposição externa da API através do kubectl proxy
- Gestão de permissões e acessos
- Configuração da rede virtual entre máquinas
- Monitorização dos recursos do cluster
- Gestão dinâmica dos recursos Kubernetes

Uma das maiores dificuldades encontradas esteve relacionada com a configuração do cluster multi-node. Durante os testes iniciais surgiram problemas de comunicação entre os worker nodes e o master node devido a configurações incorretas da rede virtual e regras de firewall.

Outro desafio importante esteve relacionado com a exposição da API Kubernetes para acesso externo através da aplicação frontend. Foi necessário configurar corretamente o kubectl proxy e validar a conectividade entre as máquinas virtuais e a aplicação desenvolvida.

Dificuldade	Descrição
Cluster Multi-node	Problemas na associação dos worker nodes
Rede Virtual	Comunicação entre máquinas virtuais
API Kubernetes	Exposição e acessibilidade da API
Frontend	Integração correta com a API REST
Monitorização	Recolha de métricas do cluster

Table 16: Principais dificuldades encontradas durante o desenvolvimento

8.2. Aspetos Positivos do Projeto

Apesar das dificuldades encontradas, o desenvolvimento do projeto trouxe várias vantagens e aprendizagens importantes.

O projeto permitiu compreender de forma prática o funcionamento de tecnologias modernas utilizadas atualmente em ambientes empresariais e infraestruturas cloud-native.

Entre os principais aspetos positivos destacam-se:

- Aprendizagem prática de Kubernetes
- Experiência com containers e Docker
- Configuração de clusters distribuídos
- Desenvolvimento de aplicações frontend modernas
- Integração entre frontend e APIs REST
- Monitorização e gestão de recursos distribuídos
- Consolidação de conhecimentos de redes e virtualização

Outro aspeto bastante positivo foi a possibilidade de desenvolver uma aplicação completamente funcional, capaz de interagir diretamente com a API Kubernetes e realizar operações reais de gestão do cluster.

A aplicação desenvolvida permite:

- Listar nodes
- Criar e eliminar namespaces
- Criar e eliminar pods
- Criar e eliminar deployments
- Criar e eliminar services
- Monitorizar recursos do cluster

Estas funcionalidades permitiram criar uma solução bastante próxima de plataformas reais de administração Kubernetes.

8.3. Qualidade da Solução Desenvolvida

A solução implementada apresentou um funcionamento estável e permitiu cumprir todos os requisitos obrigatórios definidos no enunciado do Trabalho Laboratorial.

A aplicação desenvolvida demonstrou capacidade de:

- Comunicar corretamente com a API Kubernetes
- Atualizar dinamicamente os dados apresentados
- Gerir recursos distribuídos
- Monitorizar o estado do cluster
- Disponibilizar uma interface intuitiva e organizada

Foi também dada especial atenção à experiência de utilização da aplicação, procurando desenvolver uma interface moderna, organizada e intuitiva para o utilizador.

Foram implementados:

- Menus organizados
- Navegação simplificada
- Feedback visual de sucesso e erro
- Atualização dinâmica dos recursos
- Organização clara das funcionalidades
- Interface moderna e responsiva

8.4. Melhorias Futuras

Apesar de a aplicação cumprir todos os objetivos definidos inicialmente, existem várias funcionalidades adicionais que poderiam ser implementadas futuramente para melhorar ainda mais a solução desenvolvida.

Uma das principais melhorias seria a implementação de um sistema completo de autenticação e gestão de permissões, permitindo restringir o acesso a determinadas funcionalidades da aplicação.

Outra melhoria importante seria a integração de sistemas de monitorização mais avançados, permitindo apresentar gráficos detalhados da utilização de CPU, memória RAM e tráfego de rede dos nodes e pods do cluster.

Também seria interessante implementar funcionalidades relacionadas com automação e deployment contínuo, aproximando ainda mais a aplicação de ambientes empresariais reais.

Entre as possíveis melhorias futuras destacam-se:

- Sistema de autenticação de utilizadores
- Gestão de permissões e roles
- Monitorização avançada em tempo real
- Logs centralizados
- Gestão de volumes persistentes
- Integração com Helm Charts
- Integração com Prometheus e Grafana
- Suporte para Ingress Controllers
- Sistema de notificações e alertas
- Deploy automático de aplicações
- Dashboard com gráficos avançados
- Gestão automática de backups
- Integração com CI/CD pipelines

Melhoria	Objetivo	Benefício
Autenticação	Maior segurança	Controlo de acessos
Monitorização	Mais controlo do cluster	Maior visibilidade
Logs Centralizados	Diagnóstico de erros	Facilidade de manutenção
Helm Charts	Automação de deployments	Maior rapidez
Alertas	Monitorização em tempo real	Deteção rápida de falhas
Grafana	Visualização gráfica	Melhor análise de métricas
CI/CD	Deploy automático	Maior produtividade

Table 17: Possíveis melhorias futuras

9. Conclusão

Concluindo, o presente Trabalho Laboratorial permitiu implementar com sucesso uma infraestrutura Kubernetes funcional, composta por um control-plane node e dois worker nodes, bem como desenvolver uma aplicação frontend para gestão dos recursos disponibilizados pelo cluster.

Ao longo do projeto foram analisadas diferentes soluções Kubernetes, tendo sido escolhida a solução Kind devido à sua simplicidade, desempenho e facilidade de implementação em ambientes laboratoriais.

A aplicação desenvolvida permite realizar operações fundamentais sobre recursos Kubernetes, incluindo gestão de namespaces, pods, deployments e services, para além da monitorização geral do cluster através de um dashboard central.

Este projeto contribuiu significativamente para a consolidação de conhecimentos nas áreas de Kubernetes, virtualização, APIs REST, DevOps e desenvolvimento frontend.

Para além das competências técnicas adquiridas, o projeto permitiu também desenvolver capacidades relacionadas com resolução de problemas, configuração de infraestruturas distribuídas e integração entre diferentes tecnologias.

O balanço final do trabalho é bastante positivo, representando uma experiência prática extremamente relevante para o contexto atual das Tecnologias de Informação e do mercado profissional.

10. Bibliografia

- [1] Kubernetes Official Website <https://kubernetes.io/>
- [2] Kubernetes API Reference <https://kubernetes.io/docs/reference/kubernetes-api/>
- [3] MicroK8s Official Website <https://microk8s.io/>
- [5] Kind Documentation <https://kind.sigs.k8s.io/>
- [6] K3s Documentation <https://k3s.io/>
- [7] Docker Official Website <https://www.docker.com/>
- [8] Vue.js Official Documentation <https://vuejs.org/>
- [9] Node.js Official Website <https://nodejs.org/>
- [10] Material de suporte fornecido na UC de Laboratório de Tecnologias de Informação